https://haoailab.com/cse291-s26/

CSE/DSC 291: Deep Learning Systems
Spring 2026

LLM, diffusion, and case studies

Optimizations and Parallelization

Basics

Logistics

- End of quarter evaluation (SET) released

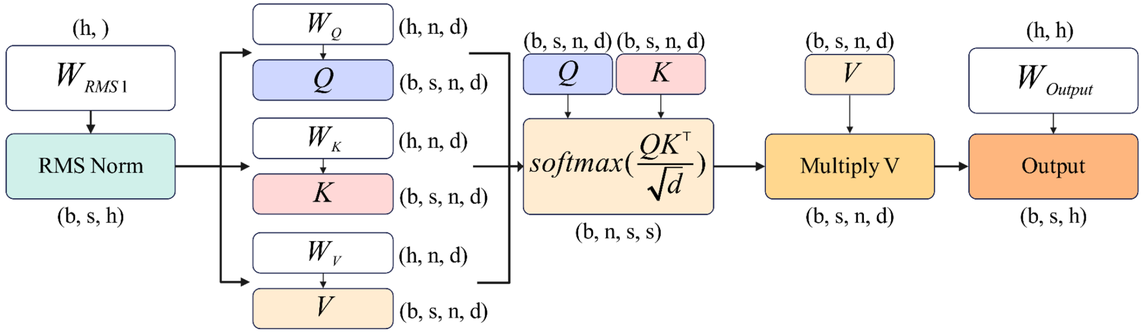
- Please fill in the evaluation
  - Again: if 80% of you filled the eval, you will get another 1%.

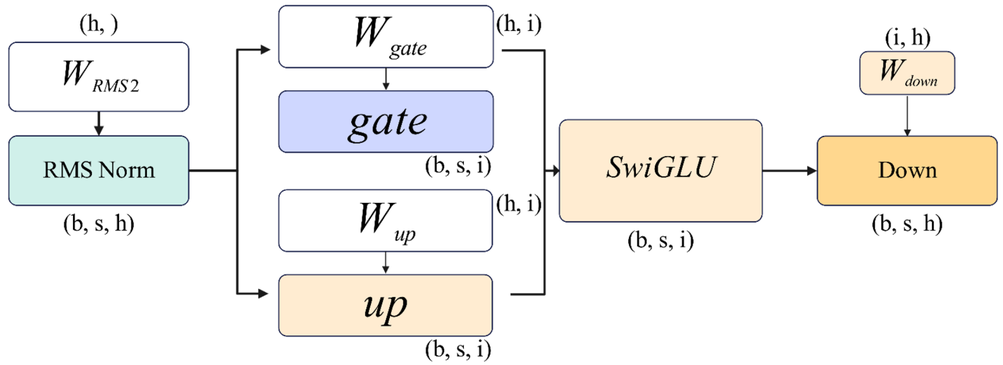
Generative LLM Inference: Autoregressive Decoding

- Pre-filling phase (0-th iteration):
  - Process *all* input tokens at once
- Decoding phase (all other iterations):
  - Process a *single* token generated from previous iteration

- Key-value cache:
  - Save attention keys and values for the following iterations to avoid recomputation
  - what is KV cache essentially?

Potential Bottleneck of LLM Inference?





- Compute:
  - Prefill: largely same with training
  - Decode: s = 1
- Memory
  - New: KV cache

Q? how about batch size b?

Serving vs. Inference

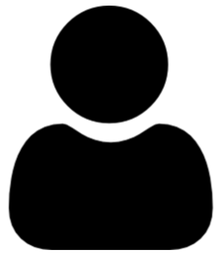
large b

Serving: many requests, online traffic, emphasize cost-per-query.

s.t. some mild latency constraints

emphasize **throughput**

b = 1

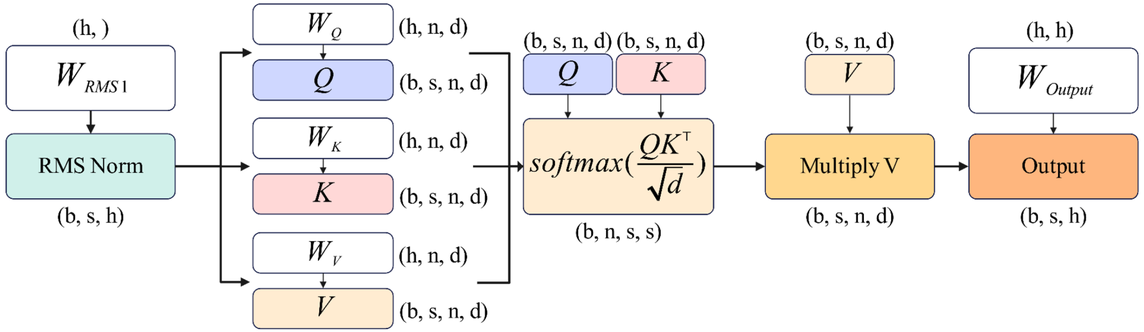


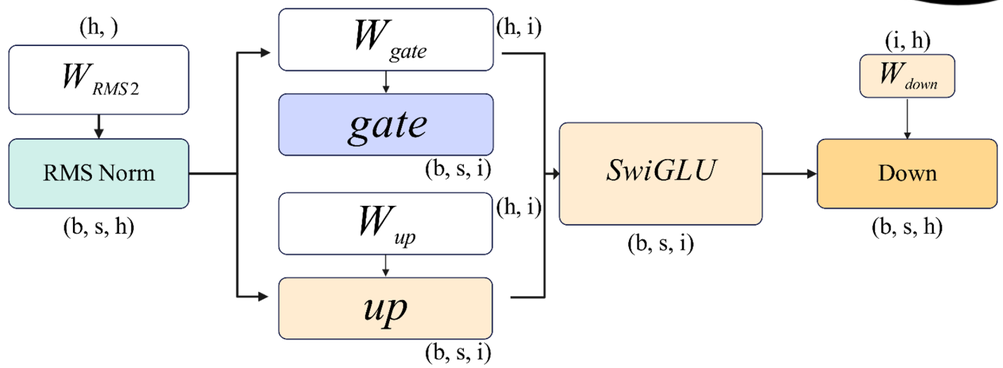
Inference: fewer request, low or offline traffic,

emphasize **latency**

large b

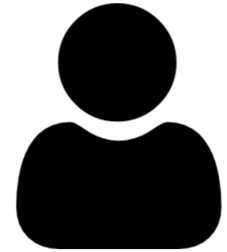
Potential Bottleneck of LLM Inference in Serving



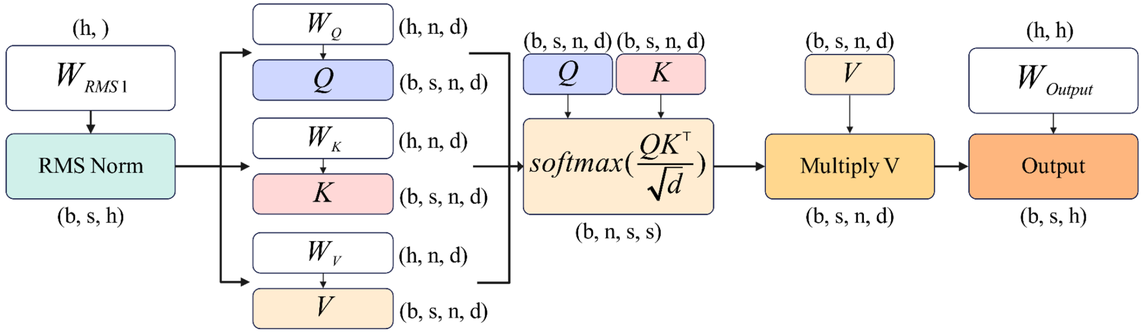


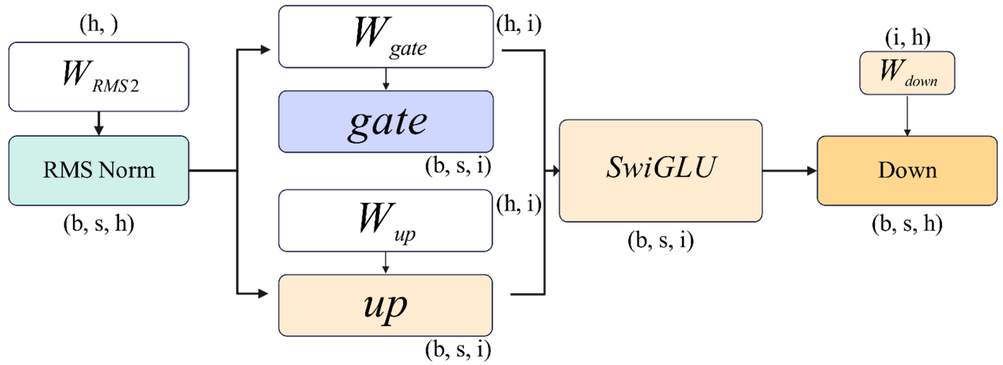
- Compute:
  - Prefill:
    - Different prompts have different length: how to batch?
  - Decode
    - Different prompts have different, unknown #generated tokens
    - s = 1, b is large
- Memory
  - New: KV cache
  - b is large -> KV is linear with b -> will KVs be large?

b=1



Potential Bottleneck of LLM Inference in Serving



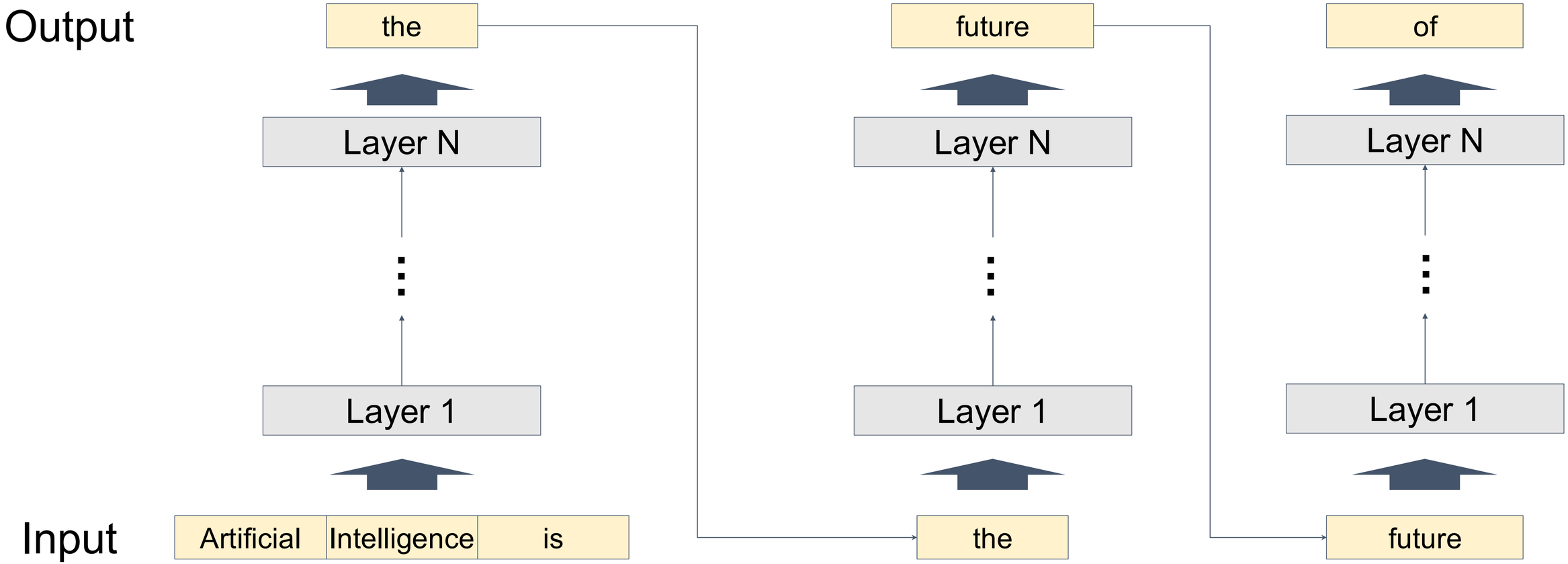


- Compute:
  - Prefill:
    - ~~Different prompts have different length: how to batch?~~
  - Decode
    - Different prompts have different, unknown #generated tokens
      - s = 1, b=1
- Memory
  - New: KV cache
    - ~~b =1 -> KV is linear with b -> will KVs be large?~~

Problems of bs = 1

max AI = #ops / #bytes

Recap: Inference process of LLMs

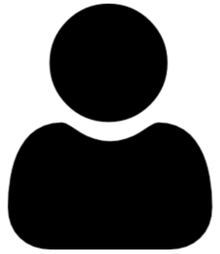


Repeat until the sequence

- Reaches its pre-defined maximum length (e.g., 2048 tokens)
- Generates certain tokens (e.g., “<|*end of sequence*|>”)

Problem of bs = 1

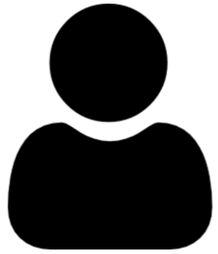
b=1



Latency = step latency * # steps

Speculative decoding reduces this, hence amortize the memory moving cost (but it may increase compute cost)

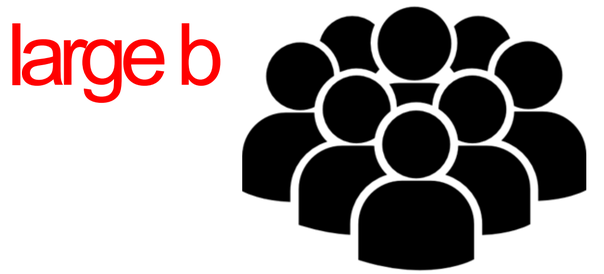
b=1



Large Language Models

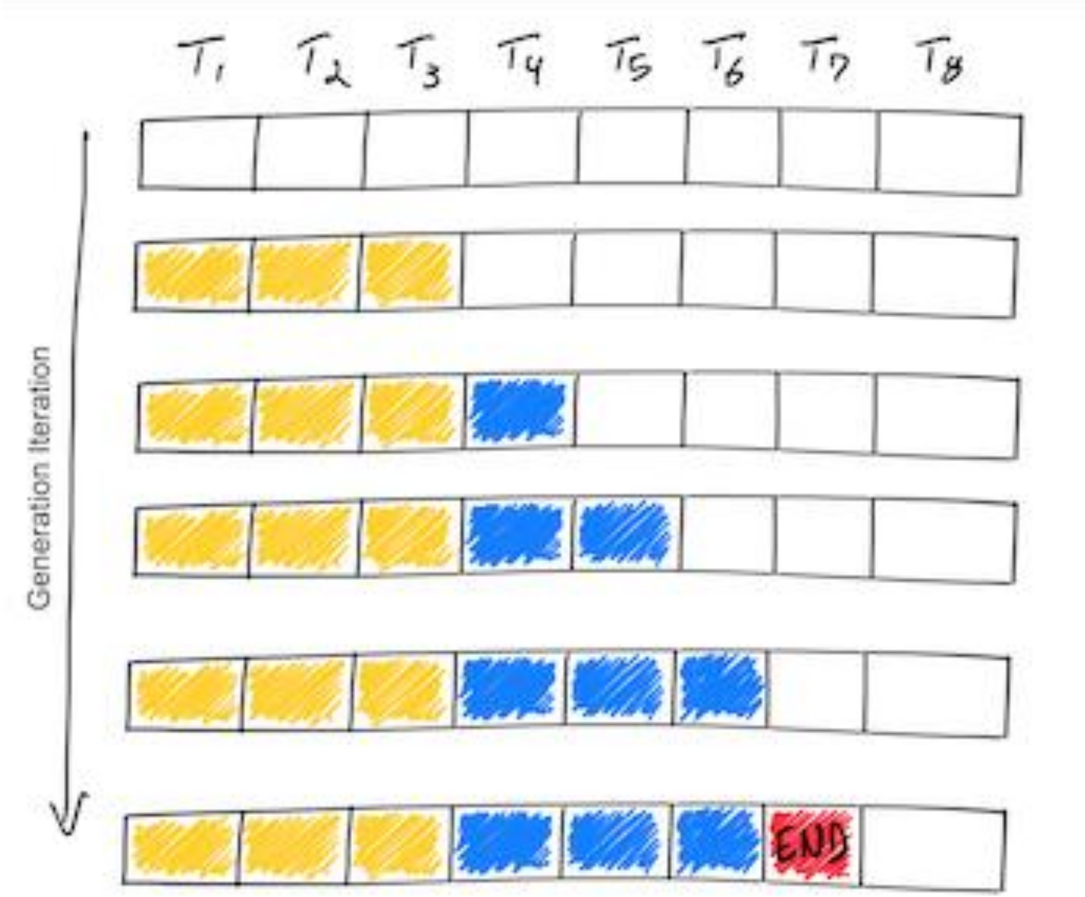
- LLM Internals
- **Scaling Law**
- Serving and inference optimization
  - Speculative decoding (question in your PA3)

Large Language Models

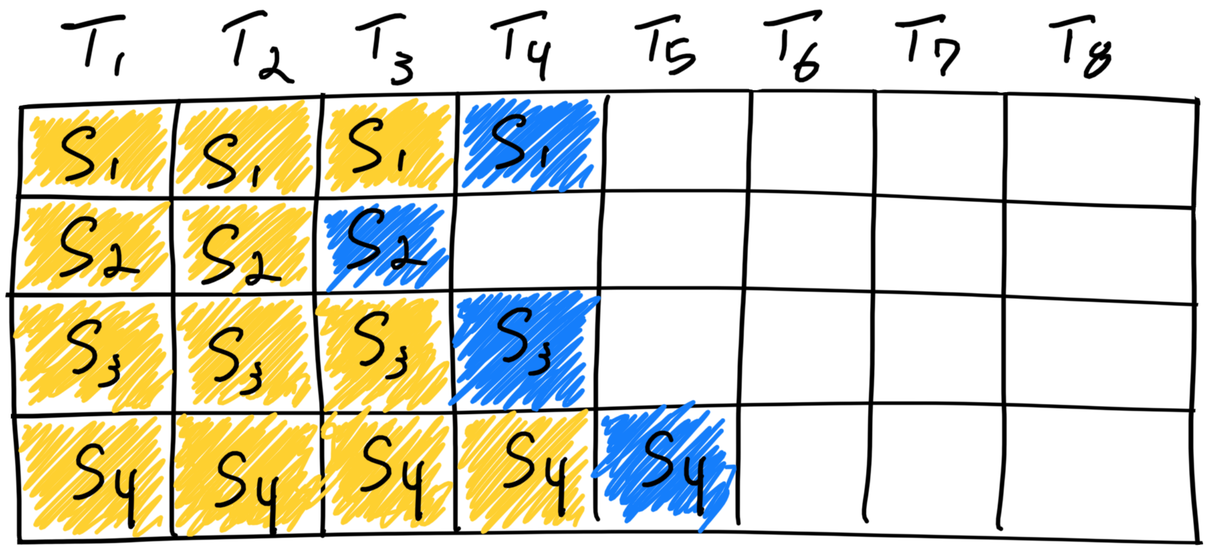


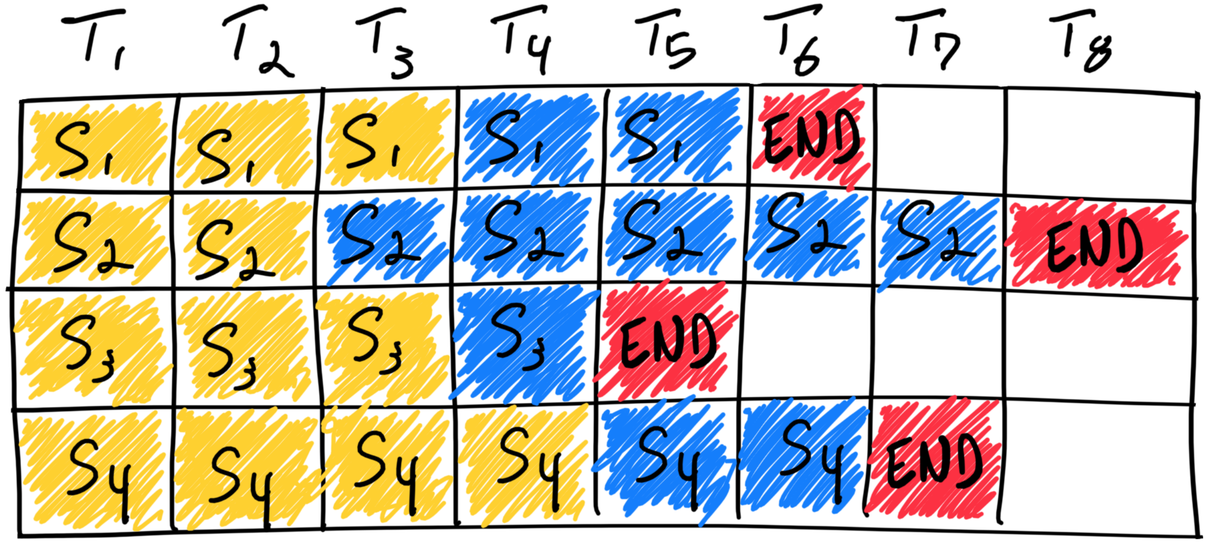
- LLM Internals
- **Scaling Law**
- Serving and inference optimization
  - Speculative decoding (question in your PA3)
  - **Continuous batching and Paged attention**

LLM Decoding Timeline



Batching Requests to Improve GPU Performance

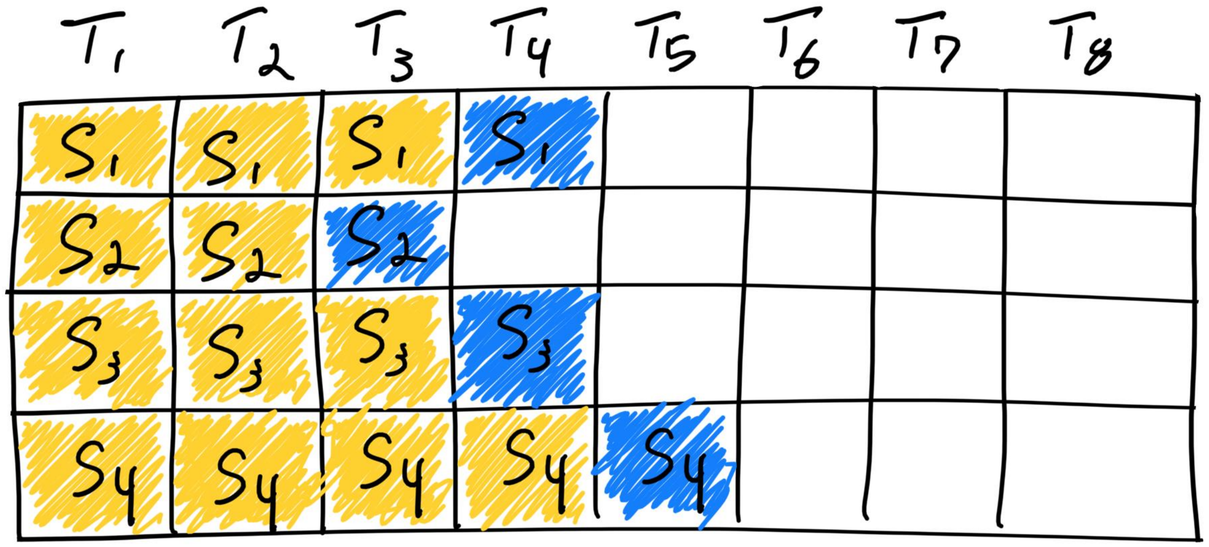


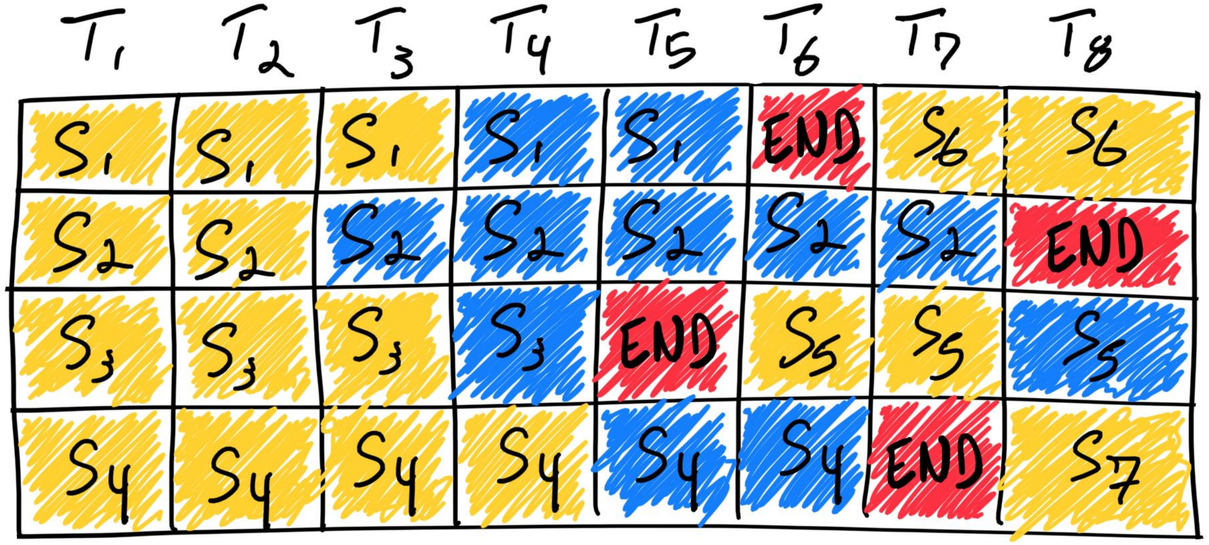


Issues with static batching:

- Requests may complete at different iterations
- Idle GPU cycles
- New requests cannot start immediately

Continuous Batching



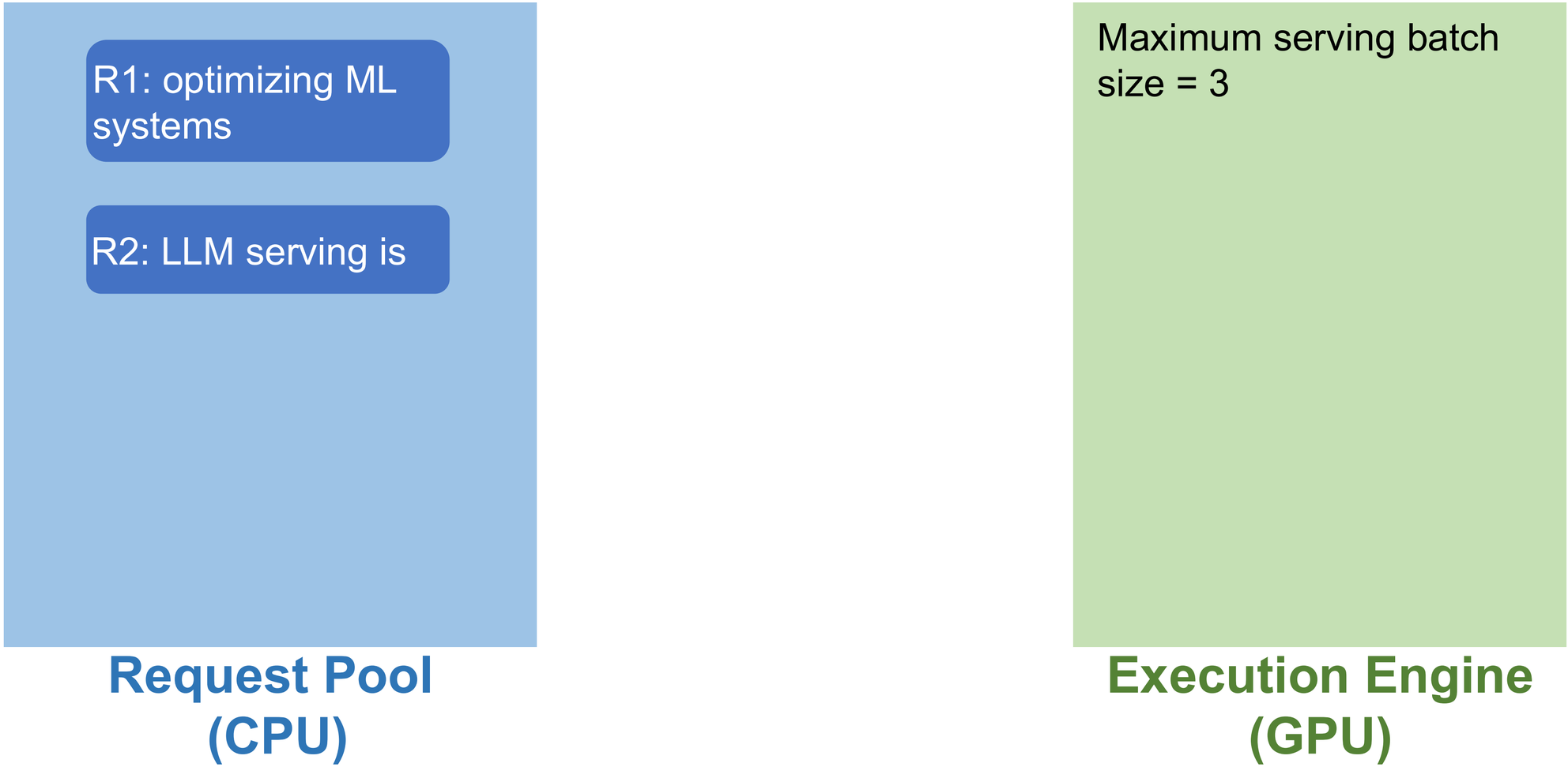


Benefits:

- Higher GPU utilization
- New requests can start immediately

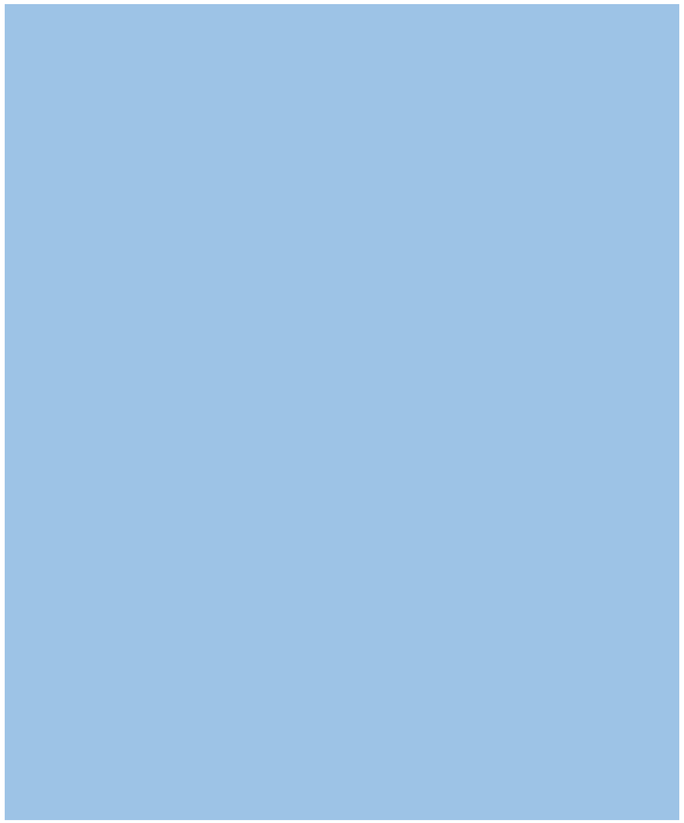
Continuous Batching Step-by-Step

- Receives two new requests R1 and R2



Continuous Batching Step-by-Step

- Iteration 1: decode R1 and R2



Request Pool (CPU)

Maximum serving batch size = 3

R1: optimizing ML systems

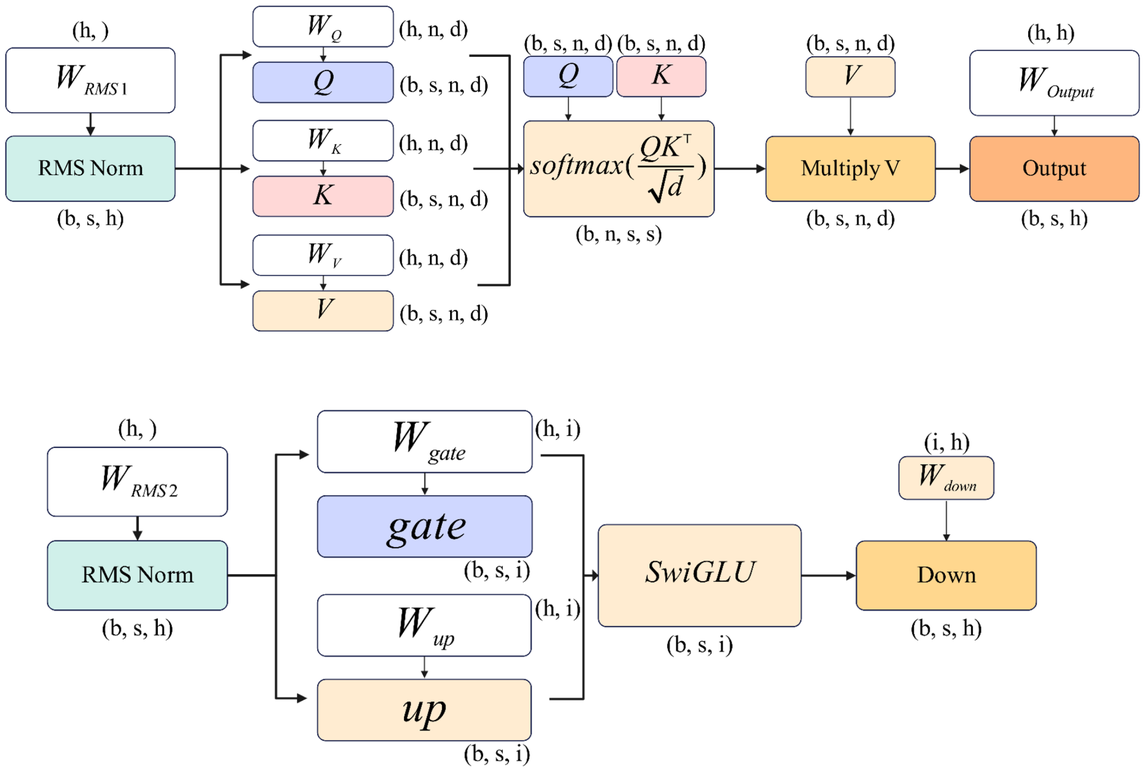
R2: LLM serving is

Execution Engine (GPU)

Iteration 1

Continuous Batching Step-by-Step

- Iteration 1: decode R1 and R2



Q: How to batch these?

Maximum serving batch size = 3

R1: optimizing ML systems

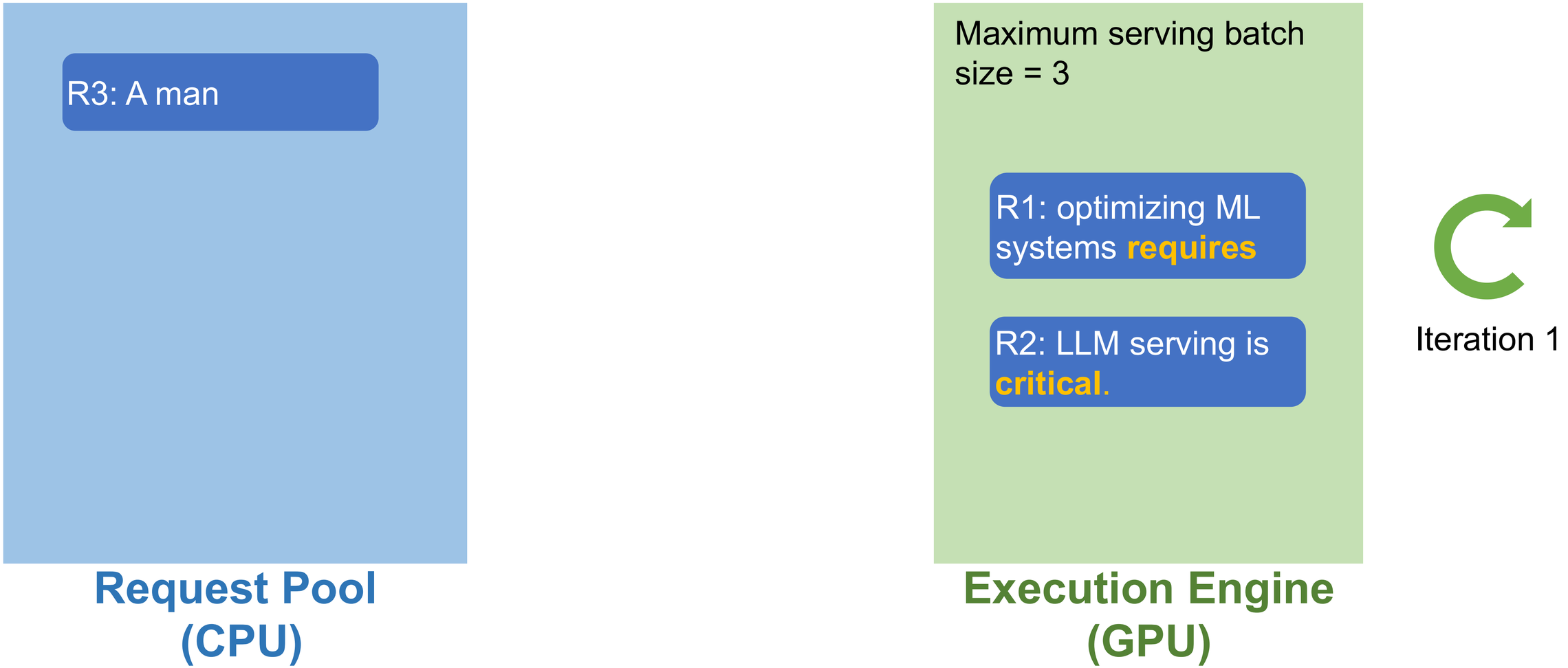
R2: LLM serving is

Iteration 1

Execution Engine (GPU)

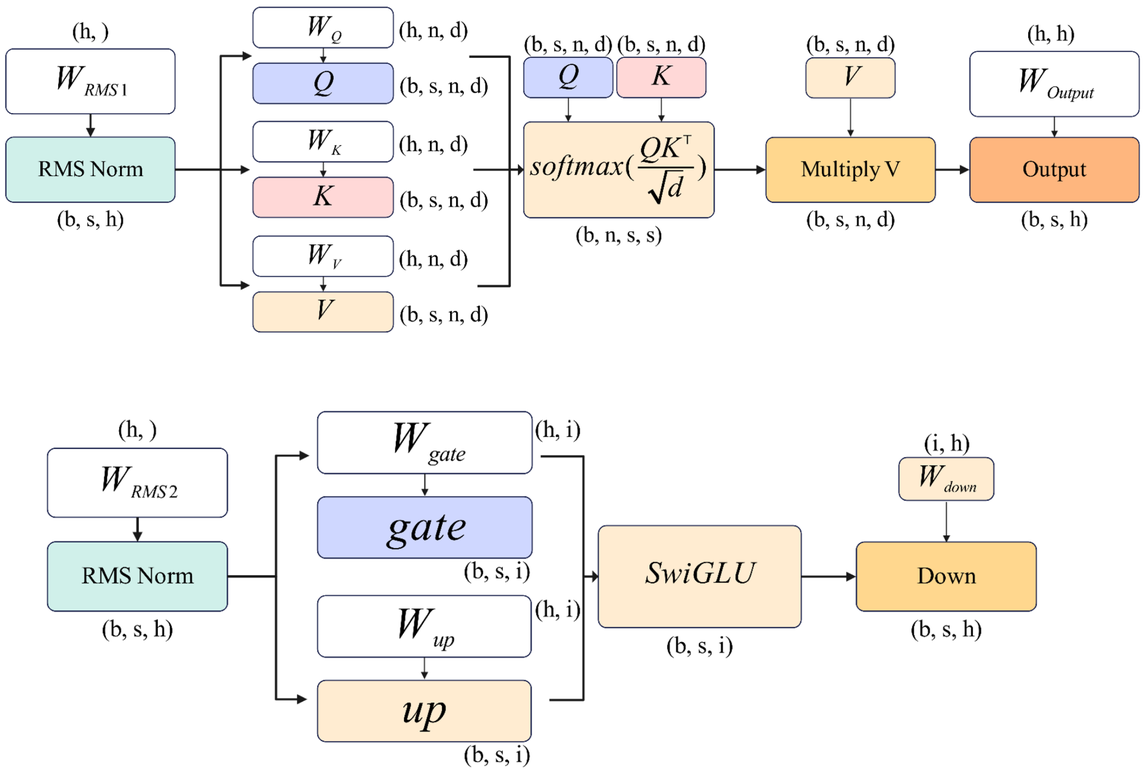
Continuous Batching Step-by-Step

- Receive a new request R3; finish decoding R1 and R2



Continuous Batching Step-by-Step

- Receive a new request R3; finish decoding R1 and R2



Q: How to batch these?

Maximum serving batch size = 3

R1: optimizing ML systems **requires**

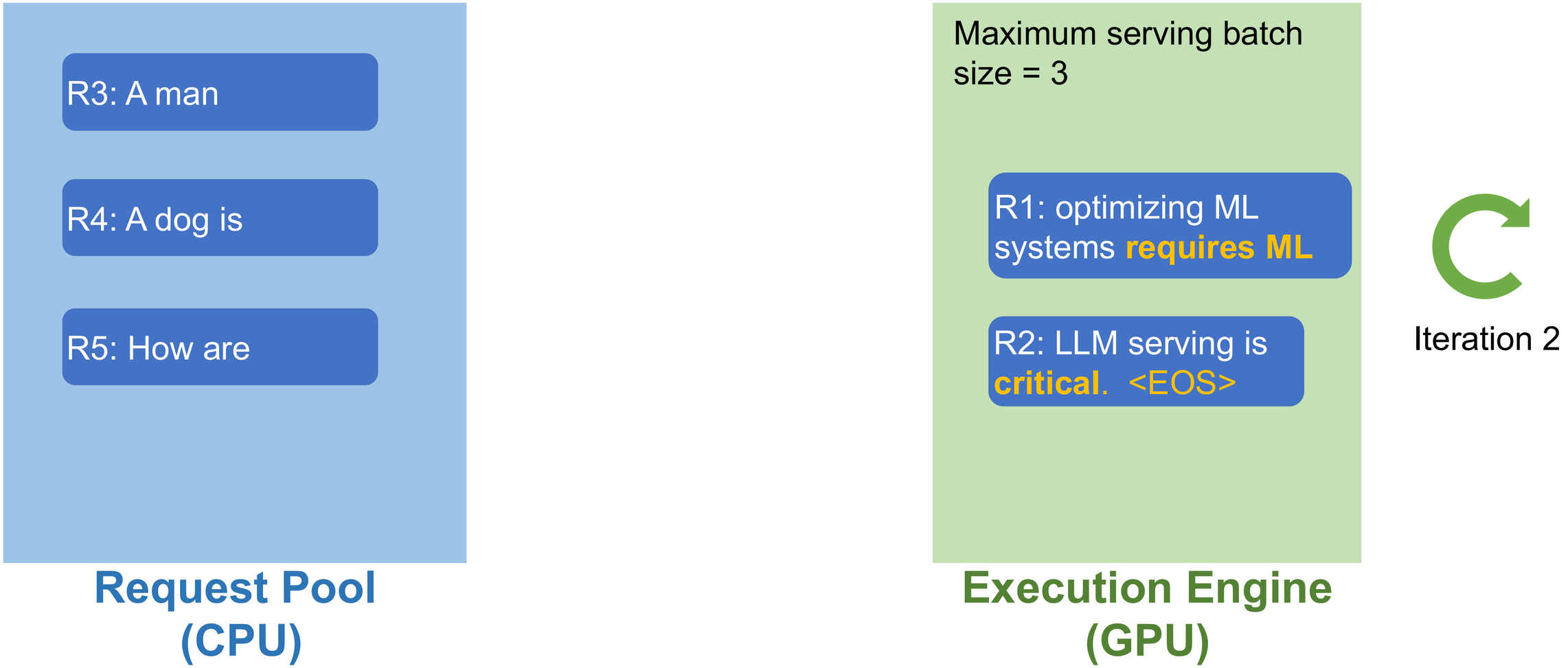
R2: LLM serving is **critical**.

Iteration 1

Execution Engine (GPU)

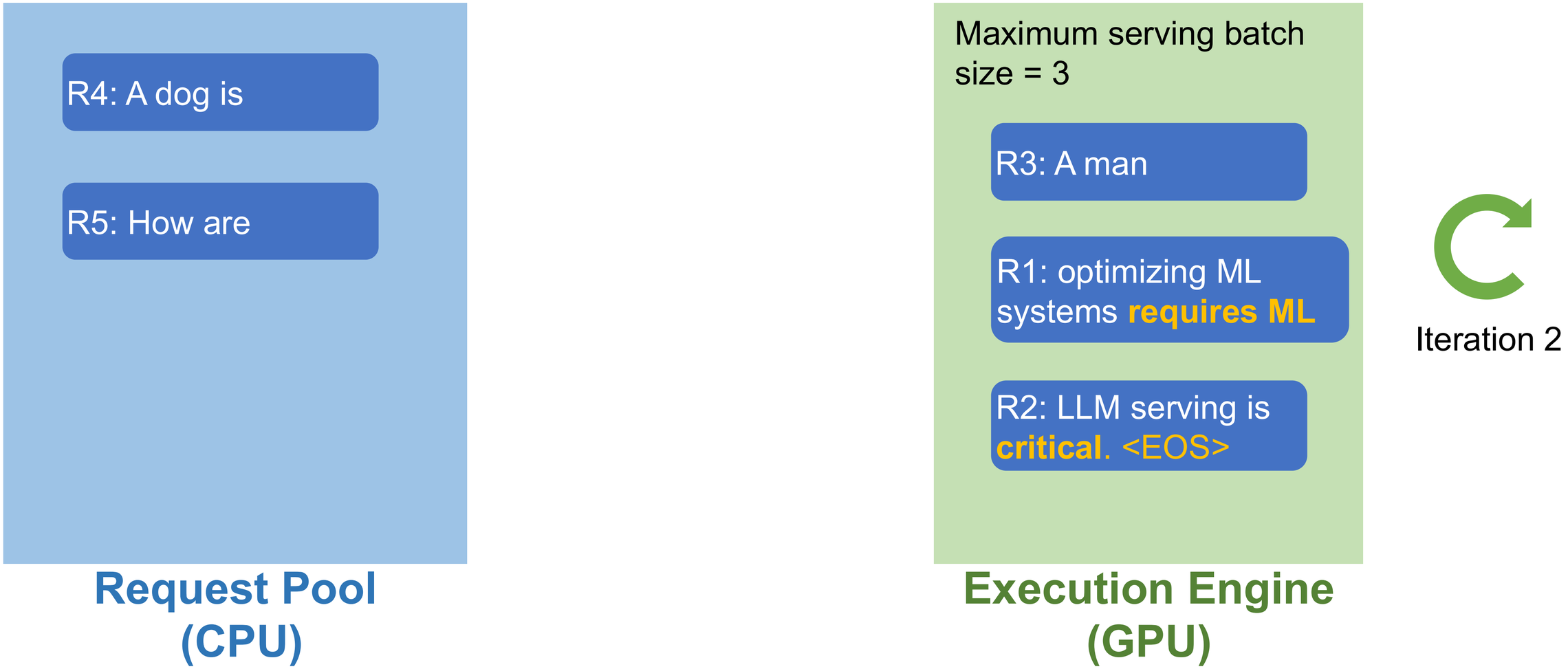
Traditional Batching

- Receive a new request R3; finish decoding R1 and R2



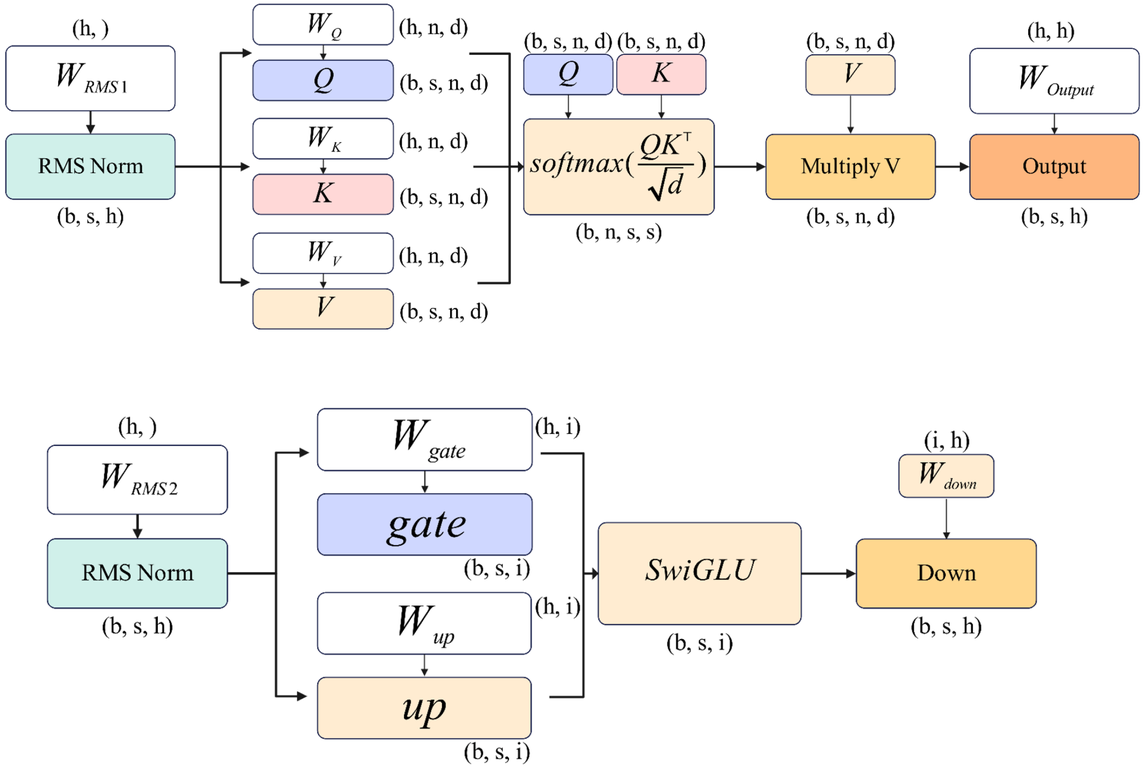
Continuous Batching

- Iteration 2: decode R1, R2, R3; receive R4, R5; R2 completes



Continuous Batching

- Iteration 2: decode R1, R2, R3; receive R4, R5; R2 completes



Q: How to batch these?

Maximum serving batch size = 3

R3: A man

R1: optimizing ML systems **requires ML**

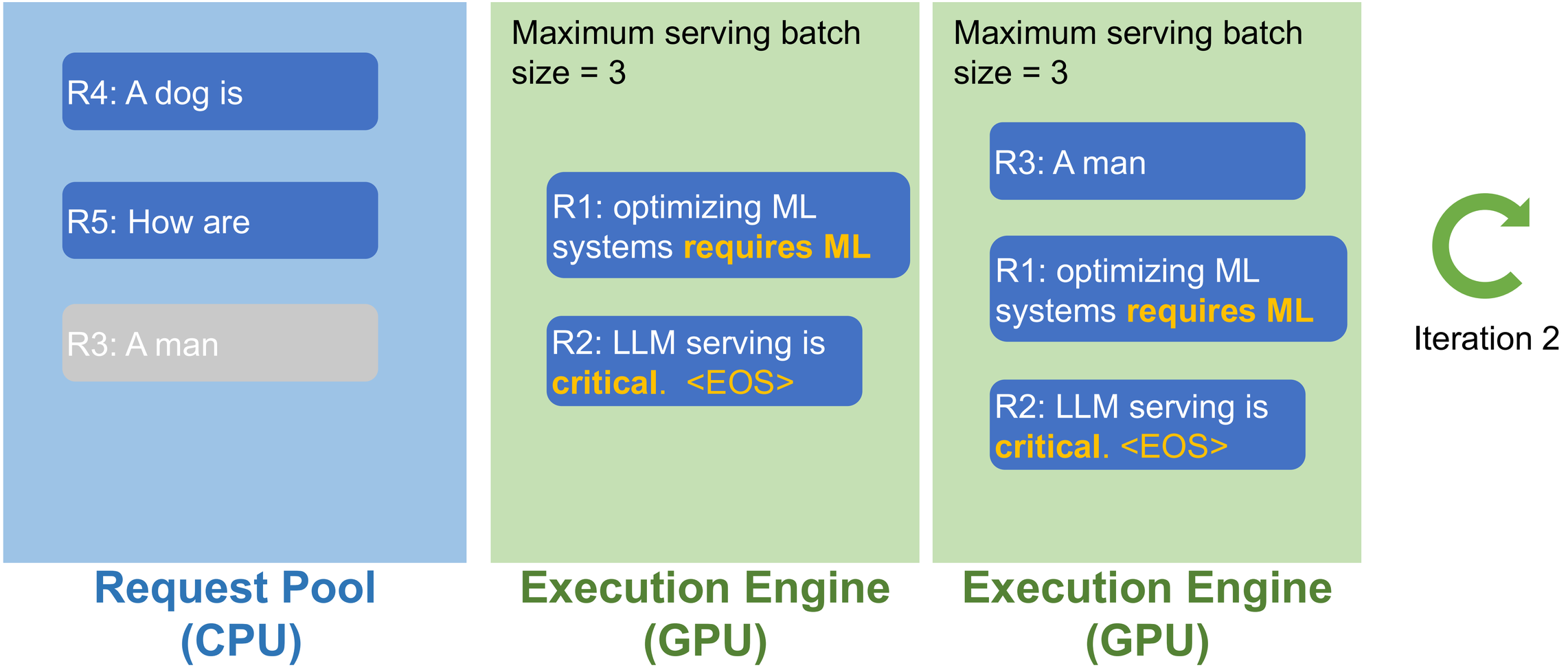
R2: LLM serving is **critical** <EOS>

Iteration 2

Execution Engine (GPU)

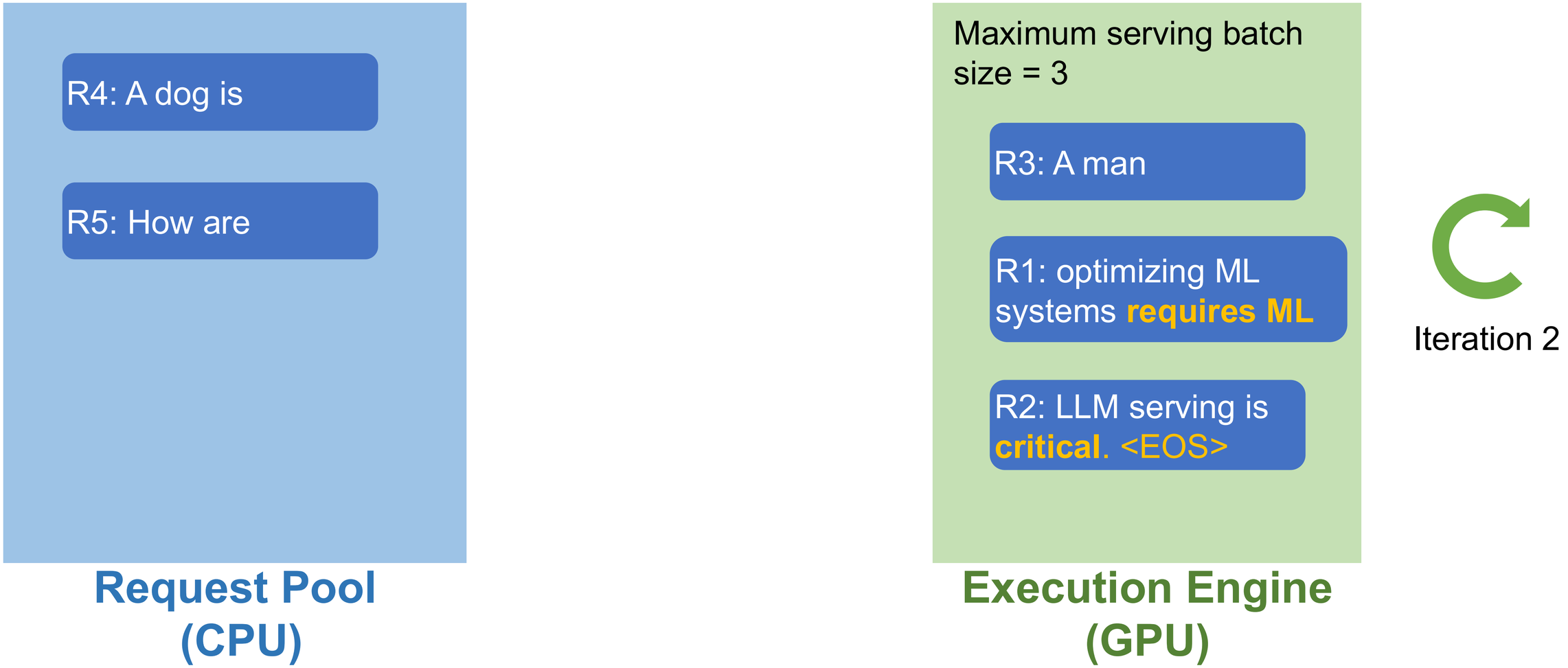
Traditional vs. Continuous Batching

- Iteration 2: decode R1, R2, R3; receive R4, R5; R2 completes



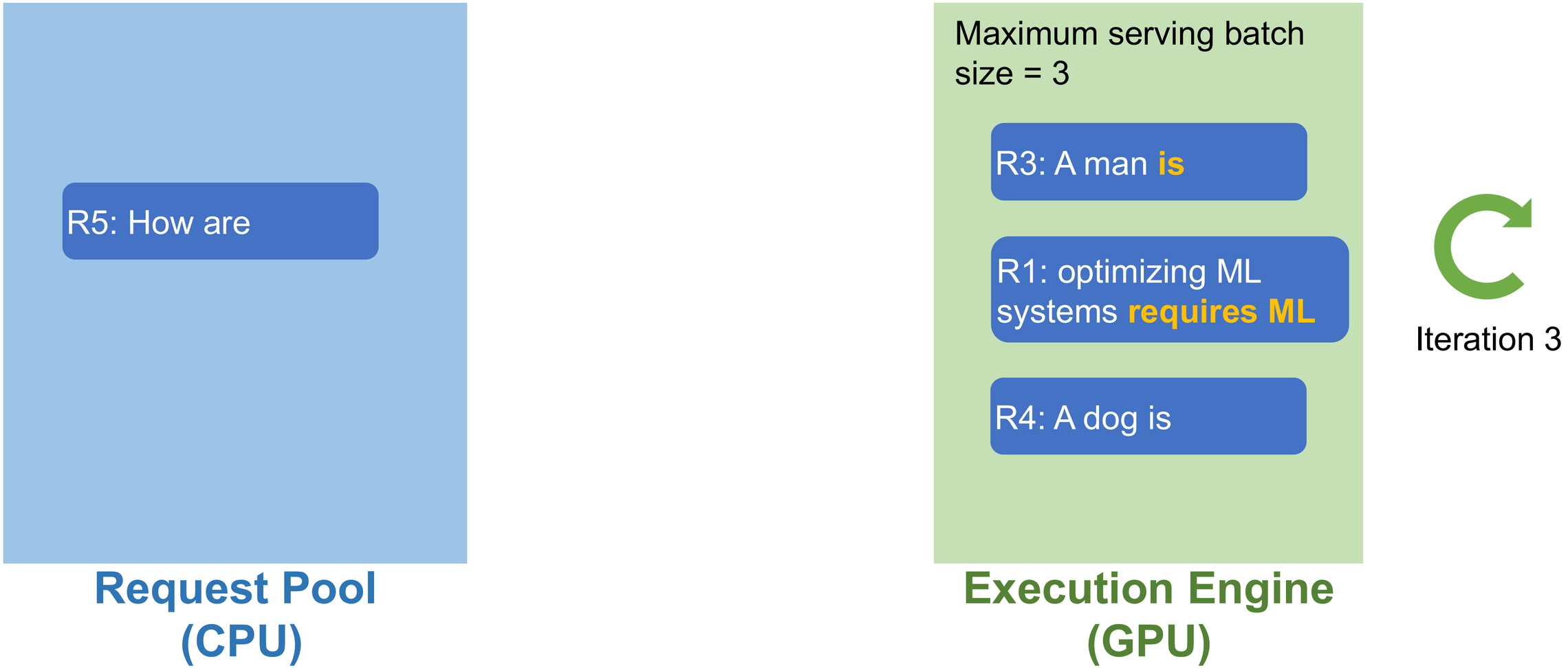
Continuous Batching

- Iteration 2: decode R1, R2, R3; receive R4, R5; R2 completes



Continuous Batching Step-by-Step

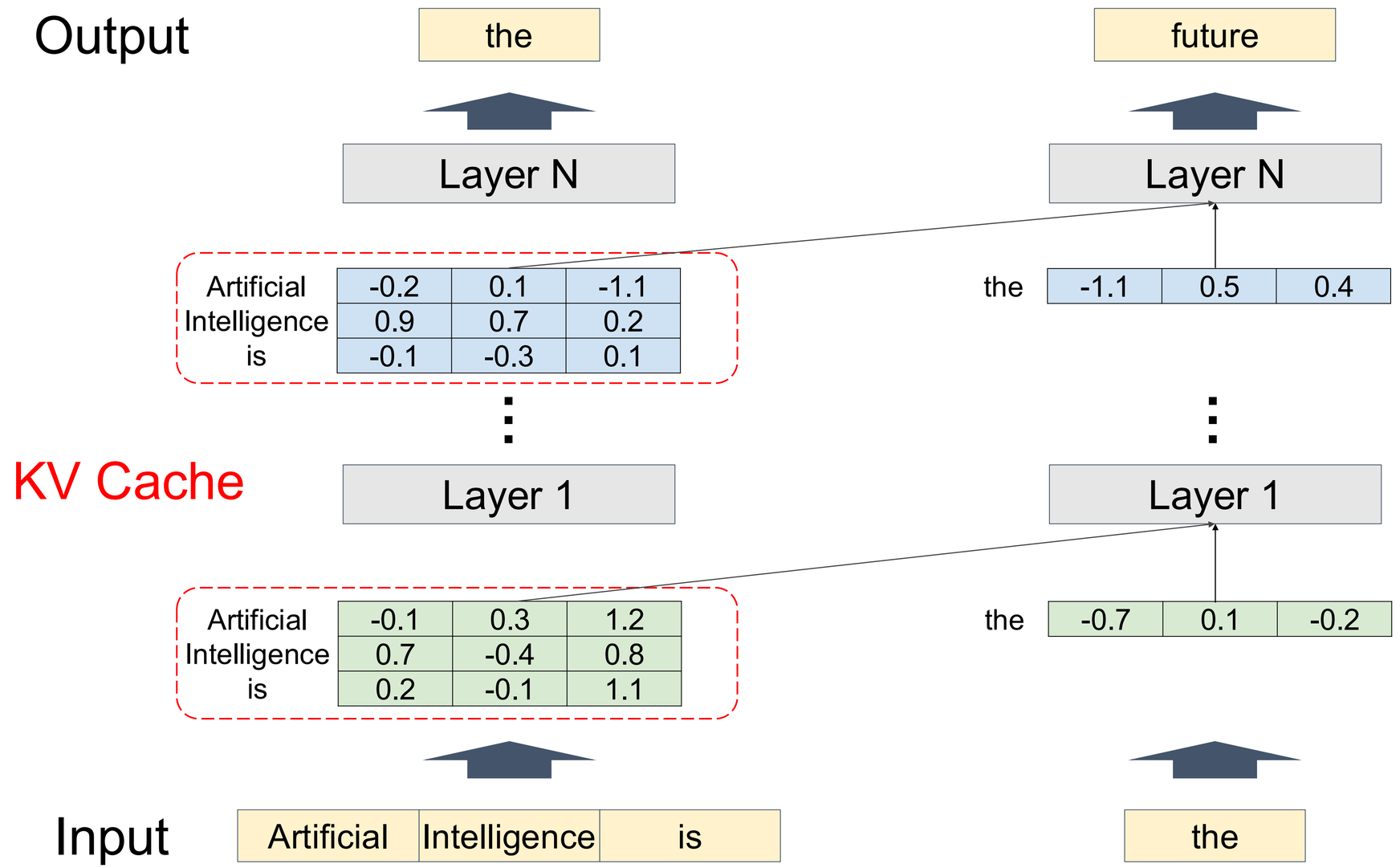
- Iteration 3: decode R1, R3, R4



Summary: Continuous Batching

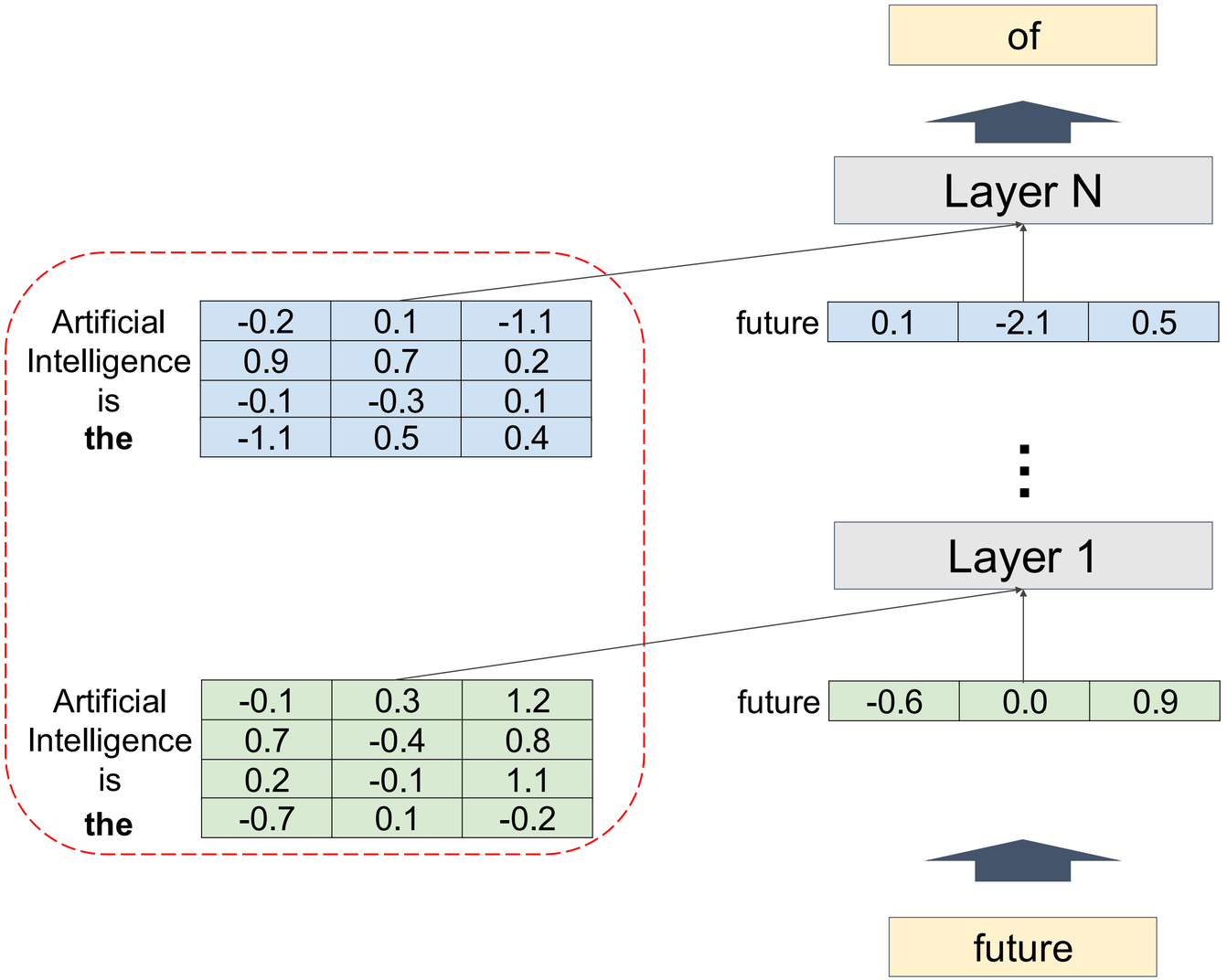
- Handle early-finished and late-arrived requests more efficiently
- Improve GPU utilization
- Key observation
  - MLP kernels are agnostic to the sequence dimension

KV Cache


Output
the
future
Layer N
Layer N
Artificial
Intelligence
is
-0.2
0.1
-1.1
0.9
0.7
0.2
-0.1
-0.3
0.1
the
-1.1
0.5
0.4
KV Cache
Layer 1
Layer 1
Artificial
Intelligence
is
-0.1
0.3
1.2
0.7
-0.4
0.8
0.2
-0.1
1.1
the
-0.7
0.1
-0.2
Input
Artificial
Intelligence
is
the

KV Cache

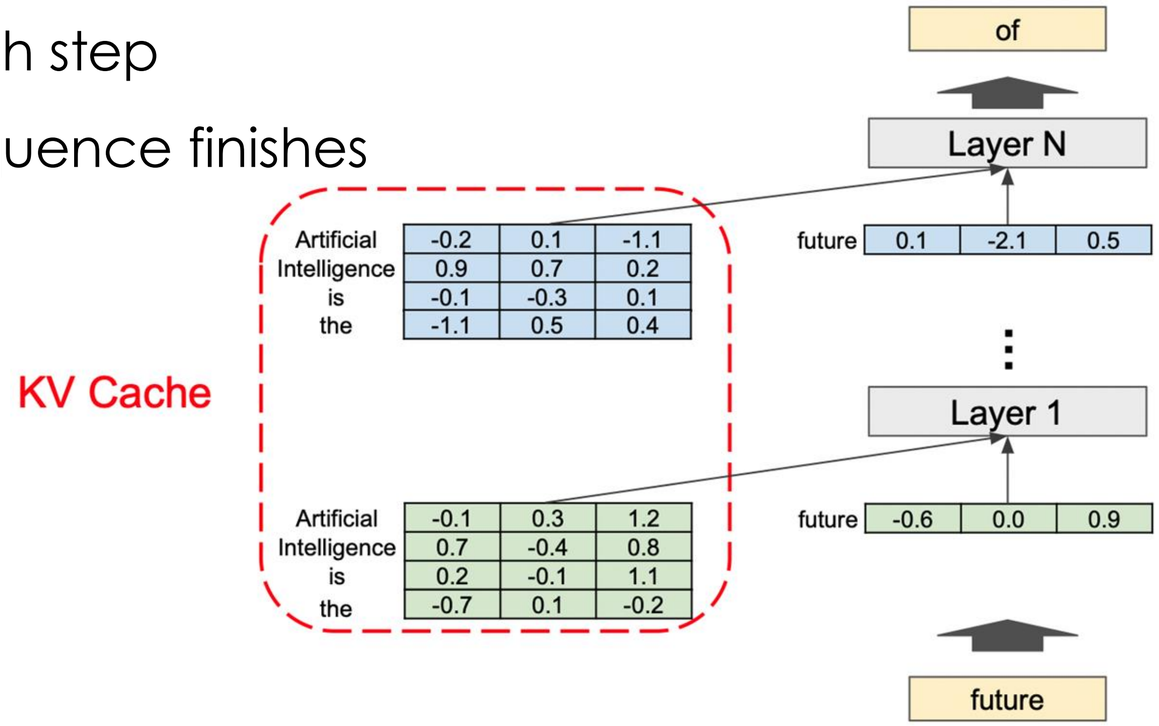
Output



Input

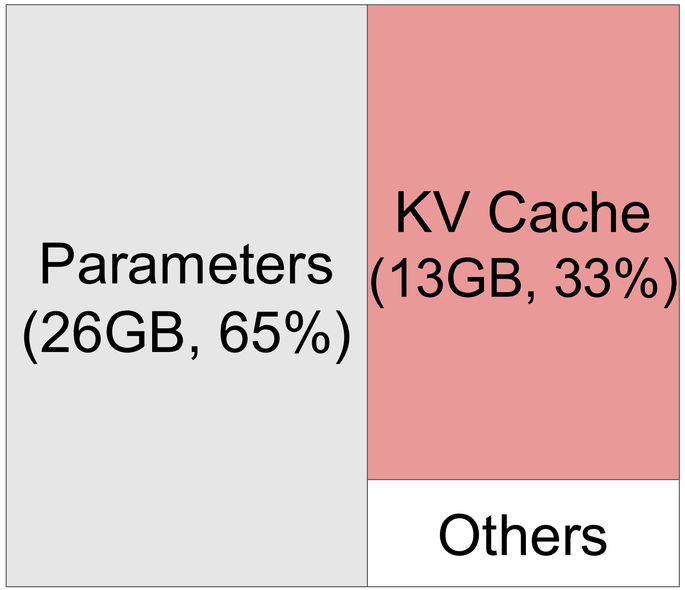
KV Cache

- Memory space to store intermediate vector representations of tokens
  - **Working set** rather than a "cache"
- The size of KV Cache dynamically grows and shrinks
  - A new token is appended in each step
  - Tokens are deleted once the sequence finishes

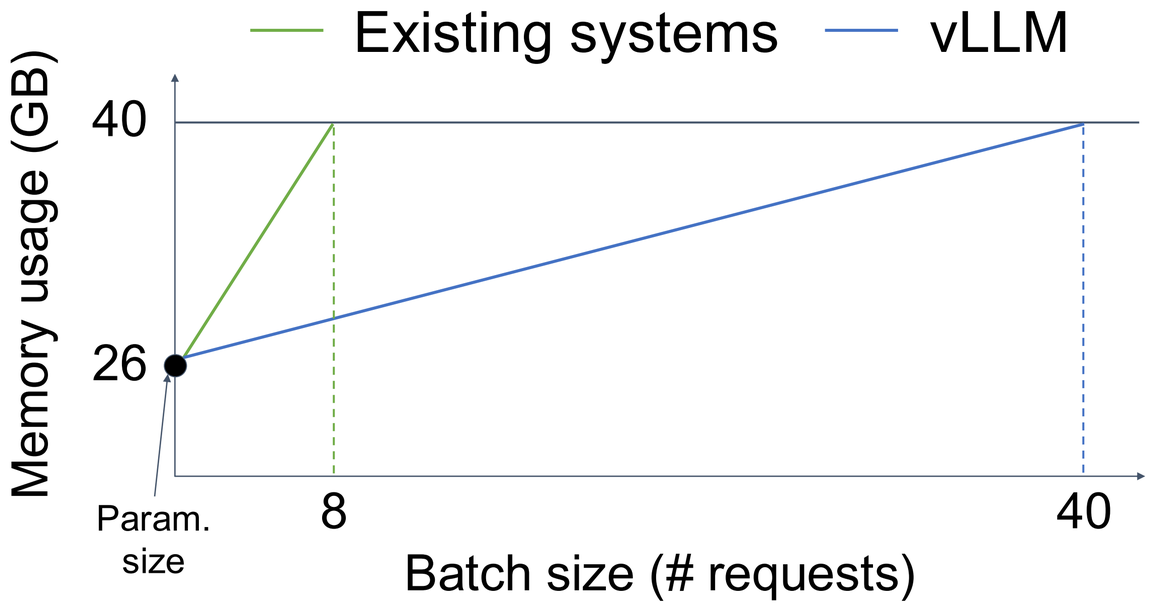


Key insight

Efficient management of KV cache is crucial for high-throughput LLM serving

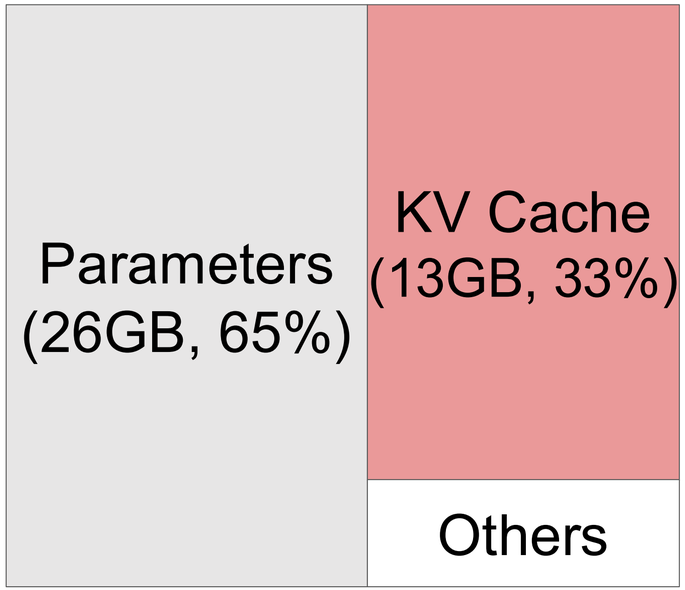


13B LLM on A100-40GB



Key insight

Efficient management of KV cache is crucial for high-throughput LLM serving



13B LLM on A100-40GB